

Laboratorium 5: Kody binarne jednoznacznie dekodowalne

Opracowanie zestawu zadań (część programistyczna)

Kacper Bednarczuk

Kod programu

```
function isUniquelyDecodable(codewords) {
  // Zamiana tablicy na zbiór dla łatwego sprawdzania przynależności
  const C = new Set(codewords);
  const S = new Set();

  // Funkcja pomocnicza: sprawdza czy str1 jest prefiksem str2
  function isPrefix(prefix, str) {
    return str.startsWith(prefix) && str !== prefix;
  }

  // Krok 1: Generowanie początkowych sufiksów z par słów kodowych
  for (const ci of C) {
    for (const cj of C) {
      if (isPrefix(ci, cj)) {
        // Pobiera sufiks napisu cj, zaczynając od pozycji ci.length
        const suffix = cj.slice(ci.length);
        // Jeśli sufiks jest słowem kodowym, kod nie jest UD
        if (C.has(suffix)) {
          return {
            isUD: false,
            reason: `Sufiks '${suffix}' jest słowem kodowym`,
          };
        }
        S.add(suffix);
      }
    }
  }

  // Jeśli nie ma żadnych sufiksów, kod jest prefiksowy (a więc UD)
  if (S.size === 0) {
    return {
      isUD: true,
      reason: "Kod prefiksowy",
    };
  }
}
```

```

}

// Iteracyjne generowanie nowych sufiksów
let previousSize = 0;
while (S.size > previousSize) {
  previousSize = S.size;
  const newSuffixes = new Set();

  // Sprawdzamy wszystkie pary: sufiks vs słowo kodowe
  for (const s of S) {
    for (const c of C) {
      // Przypadek 1: sufiks jest prefiksem słowa kodowego
      if (isPrefix(s, c)) {
        const newSuffix = c.slice(s.length);
        if (C.has(newSuffix)) {
          return {
            isUD: false,
            reason: `Sufiks '${newSuffix}' jest słowem kodowym`,
          };
        }
        newSuffixes.add(newSuffix);
      }

      // Przypadek 2: słowo kodowe jest prefiksem sufiksu
      if (isPrefix(c, s)) {
        const newSuffix = s.slice(c.length);
        if (C.has(newSuffix)) {
          return {
            isUD: false,
            reason: `Sufiks '${newSuffix}' jest słowem kodowym`,
          };
        }
        newSuffixes.add(newSuffix);
      }
    }
  }

  // Dodajemy nowe sufiksy do zbioru S
  for (const suffix of newSuffixes) {
    S.add(suffix);
  }
}

// Osiągnięto punkt stały – kod jest UD
return {
  isUD: true,
  reason: "Osiągnięto punkt stały bez konfliktów",
};
}

```

```
function testCode(name, codewords) {
  console.log(`\n=== Test: ${name} ===`);
  console.log(`Słowa kodowe: ${codewords.join(", ")}`);

  const result = isUniquelyDecodable(codewords);

  if (result.isUD) {
    console.log(`✓ Kod JEDNOZNACZNIE DEKODOWALNY`);
    console.log(`Powód: ${result.reason}`);
  } else {
    console.log(`✗ Kod NIE JEST JEDNOZNACZNIE DEKODOWALNY`);
    console.log(`Powód: ${result.reason}`);
  }
}

console.log("WERYFIKACJA KODÓW Z ZADANIA 1.1");
testCode("Kod A", ["1", "01", "001", "000"]);
testCode("Kod B", ["00", "11", "010"]);
testCode("Kod C", ["1", "10", "001", "011"]);
console.log("\nWERYFIKACJA KODÓW Z ZADANIA 1.2");
testCode('Kod B + "01"', ["00", "11", "010", "01"]);
testCode('Kod B + "110"', ["00", "11", "010", "110"]);
testCode('Kod B + "011"', ["00", "11", "010", "011"]);
```

Analiza działania programu i wyników

Program implementuje **algorytm Sardinas-Pattersona** do weryfikacji, czy dany kod jest jednoznacznie dekodowalny. Algorytm działa w następujący sposób:

- Inicjalizacja:** Tworzony jest zbiór słów kodowych C oraz pusty zbiór "wiszących sufiksów" S .
- Krok 1 (Sufiksy początkowe):** Algorytm wyszukuje wszystkie pary słów kodowych (c_i, c_j) , w których c_i jest prefiksem c_j . Sufiks c_j (część pozostająca po odcięciu c_i) jest dodawany do zbioru S . Jeśli którykolwiek z tych początkowych sufiksów sam jest słowem kodowym, algorytm natychmiast kończy działanie, stwierdzając, że kod nie jest jednoznacznie dekodowalny. Jeśli po tym kroku zbiór S jest pusty, oznacza to, że żadne słowo kodowe nie jest prefiksem innego, a więc kod jest **kodem prefiksowym** i jest jednoznacznie dekodowalny.
- Krok 2 (Iteracja):** Jeśli zbiór S nie jest pusty, algorytm iteracyjnie generuje nowe sufiksy. W każdej iteracji porównuje każdy istniejący sufiks s ze zbioru S z każdym słowem kodowym c ze zbioru C :
 - Jeśli s jest prefiksem c , generowany jest nowy sufiks (część c po odcięciu s).
 - Jeśli c jest prefiksem s , generowany jest nowy sufiks (część s po odcięciu c).
- Warunek stopu:**

- Jeśli którykolwiek nowo wygenerowany sufix okaże się być słowem kodowym ze zbioru C , algorytm kończy działanie, a kod **nie jest** jednoznacznie dekodowalny.
- Nowe, unikalne sufiksy są dodawane do zbioru S . Proces iteracyjny jest powtarzany tak długo, jak w każdej iteracji udaje się znaleźć choć jeden nowy sufix (tj. tak długo, jak zbiór S rośnie).
- Jeśli algorytm osiągnie ten "punkt stały" (zbiór S przestaje rosnąć) bez znalezienia konfliktu, oznacza to, że kod **jest** jednoznacznie dekodowalny.

=== Test: Kod A ===

Słowa kodowe: {1, 01, 001, 000}

✓ Kod JEDNOZNACZNIE DEKODOWALNY

Powód: Kod prefiksowy

=== Test: Kod B ===

Słowa kodowe: {00, 11, 010}

✓ Kod JEDNOZNACZNIE DEKODOWALNY

Powód: Kod prefiksowy

=== Test: Kod C ===

Słowa kodowe: {1, 10, 001, 011}

X Kod NIE JEST JEDNOZNACZNIE DEKODOWALNY

Powód: Sufiks '1' jest słowem kodowym

WERYFIKACJA KODÓW Z ZADANIA 1.2

=== Test: Kod B + "01" ===

Słowa kodowe: {00, 11, 010, 01}

✓ Kod JEDNOZNACZNIE DEKODOWALNY

Powód: Osiągnięto punkt stały bez konfliktów

=== Test: Kod B + "110" ===

Słowa kodowe: {00, 11, 010, 110}

✓ Kod JEDNOZNACZNIE DEKODOWALNY

Powód: Osiągnięto punkt stały bez konfliktów

=== Test: Kod B + "011" ===

Słowa kodowe: {00, 11, 010, 011}

✓ Kod JEDNOZNACZNIE DEKODOWALNY

Powód: Kod prefiksowy

Wniosek: wyniki testów z programu pokrywają się z ręcznymi obliczeniami przeprowadzonymi w części teoretycznej.